

Topic -3: Finite Markov Decision Processes

AIMLCZG512-Deep Reinforcement Learning

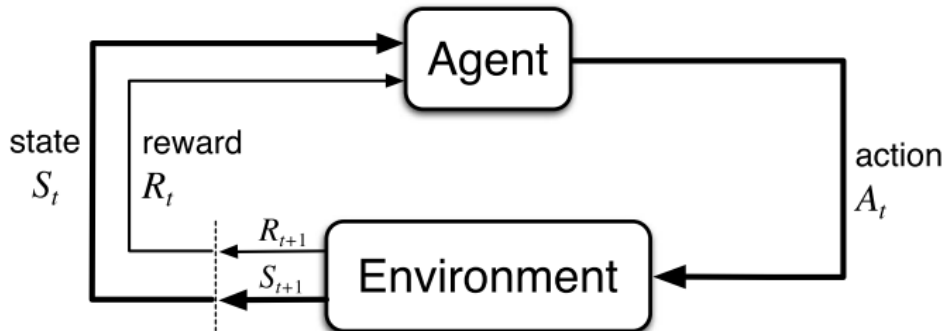
Lecture Notes

1. Topics Covered

1. Agent-environment interaction and the finite MDP formulation.
2. Environment dynamics using transition probabilities and expected rewards.
3. Agent-environment boundary and its role in modelling.
4. Formulating selected RL tasks as finite MDPs.
5. Goals, rewards, returns, and episodic/continuing tasks.
6. Policies, state-value functions, action-value functions, and Bellman equations.
7. Gridworld example and interpretation of value functions.
8. Bellman optimality equations, backup diagrams, and approximation issues.

2. Agent-Environment Interface

Finite Markov decision processes (finite MDPs) provide the standard formal framework for modelling **sequential decision making** in reinforcement learning. Unlike a bandit problem, an MDP has a state that can change after each action. Therefore, an action can affect not only the immediate reward, but also the next situation and the rewards available later.



Agent-environment interaction: at time t , the agent observes S_t , selects A_t , then receives R_{t+1} and S_{t+1} .

At each discrete time step $t = 0, 1, 2, \dots$, the agent receives a state $S_t \in \mathcal{S}$ and selects an action $A_t \in \mathcal{A}(S_t)$. One time step later, the environment returns a reward $R_{t+1} \in \mathcal{R}$ and a next state S_{t+1} . This produces the trajectory

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \dots$$

A **finite MDP** is an MDP in which the sets of states, actions, and rewards are finite. This finite setting is important because policies and value functions can be represented as tables, and the central equations can be written as finite sums.

2.1. Elements of a Finite MDP

Element	Meaning in an MDP
State set, $\mathcal{S}$	The finite set of possible situations in which the agent may find itself.
Action set, $\mathcal{A}(s)$	The finite set of actions available in state s .
Reward set, $\mathcal{R}$	The finite set of possible numerical reward values.
Dynamics, $p(s', r \mid s, a)$	The probability of receiving reward r and moving to next state s' after taking action a in state s .
Policy, $\pi(a \mid s)$	The agent's action-selection rule: the probability of choosing action a in state s .
Discount factor, γ	A parameter in $[0, 1]$ that controls how strongly future rewards are valued.

The Markov property means that the distribution of the next state and reward depends on the present state and action, not on the full earlier history once the present state is known. Thus, the state must contain the information from the past that is relevant for predicting future consequences.

3. Dynamics of a Finite MDP

The dynamics of a finite MDP specify how the environment responds to actions. They define a probability distribution over the next state and reward for every current state-action pair.

3.1. Transition and Reward Dynamics

For all $s, s' \in \mathcal{S}$, $a \in \mathcal{A}(s)$, and $r \in \mathcal{R}$, the one-step dynamics are

$$p(s', r \mid s, a) \doteq \Pr\{S_t = s', R_t = r \mid S_{t-1} = s, A_{t-1} = a\}. \quad (1)$$

Equation (1) says: if the agent was in state s and took action a , the environment may move to state s' and give reward r with probability $p(s', r \mid s, a)$.

For each fixed s and a , the probabilities over all possible next states and rewards must sum to one:

$$\sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a) = 1, \quad \forall s \in \mathcal{S}, a \in \mathcal{A}(s). \quad (2)$$

Equation (2) simply says that one of the possible next-state/reward outcomes must occur.

The state-transition probability is obtained by summing over all possible rewards:

$$p(s' \mid s, a) \doteq \Pr\{S_t = s' \mid S_{t-1} = s, A_{t-1} = a\} = \sum_{r \in \mathcal{R}} p(s', r \mid s, a). \quad (3)$$

Equation (3) gives only the probability of the next state, without retaining which reward value occurred.

The expected reward for a state-action pair is

$$r(s, a) \doteq \mathbb{E}[R_t \mid S_{t-1} = s, A_{t-1} = a] = \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} p(s', r \mid s, a). \quad (4)$$

Equation (4) is the average immediate reward expected after taking action a in state s .

The expected reward for a transition from s to s' under action a is

$$r(s, a, s') \doteq \mathbb{E}[R_t \mid S_{t-1} = s, A_{t-1} = a, S_t = s'] = \sum_{r \in \mathcal{R}} r \frac{p(s', r \mid s, a)}{p(s' \mid s, a)}. \quad (5)$$

Equation (5) is useful when the next state is already known and we want the expected reward associated with that particular transition.

The dynamics can be used for prediction, planning, and analysis. If the dynamics are known, one can compute expected returns and value functions. If they are unknown, the agent must learn from sampled interaction with the environment.

4. Agent-Environment Boundary

The agent-environment boundary specifies what is controlled by the agent and what is treated as part of the environment. The agent chooses actions. The environment includes the consequences of those actions, the state transitions, and the reward generation mechanism.

The boundary is not always a physical boundary. In a robot, motors and sensors may physically belong to the robot, but they are often treated as part of the environment if the learning agent only sends high-level commands to them. In driving, actions could be defined as steering and braking, tire torques, muscle movements, or high-level route choices. The correct boundary depends on the modelling purpose.

Task	Possible agent action	Environment includes
Robot arm	Desired joint torques or target positions	Motors, link dynamics, sensors, objects, and task reward computation.
Traffic signal control	Choose or extend a traffic phase	Vehicle arrivals, queues, driver behaviour, and measured delays.
Video game agent	Joystick/button action	Game emulator, screen update, score change, and terminal condition.

The boundary represents the limit of the agent's direct control, not the limit of its knowledge. The agent may know some facts about the environment, but anything it cannot arbitrarily change is still modelled outside the agent.

5. MDP Formulations of Reinforcement Learning Tasks

An MDP formulation should identify states, actions, rewards, and transition behaviour. The following examples show how this modelling step is done.

5.1. Recycling Robot

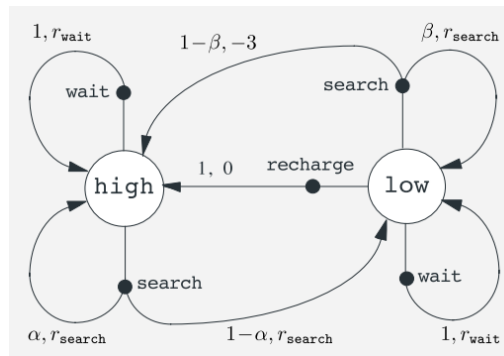
The recycling robot has two battery states: *high* and *low*. It may search for cans, wait, or recharge depending on the battery level. This example is small, but it clearly shows stochastic transitions and rewards.



Recycling robot illustration.

s	a	s'	$p(s' s, a)$	$r(s, a, s')$
high	search	high	α	r_{search}
high	search	low	$1 - \alpha$	r_{search}
low	search	high	$1 - \beta$	-3
low	search	low	β	r_{search}
high	wait	high	1	r_{wait}
high	wait	low	0	-
low	wait	high	0	-
low	wait	low	1	r_{wait}
low	recharge	high	1	0
low	recharge	low	0	-

Transition probabilities and rewards for the recycling robot.



Transition diagram for the recycling robot MDP.

- **States:** $\mathcal{S} = \{high, low\}$, representing the battery level.
- **Actions:** search, wait, and recharge, with recharge meaningful when the battery is low.
- **Rewards:** r_{search} for successful searching, r_{wait} for waiting, 0 for recharging, and -3 when searching with low battery causes failure.
- **Transitions:** parameters α and β describe the probability of battery states after searching. For example, from high and search, the robot remains high with probability α and becomes low with probability $1 - \alpha$.

This example is useful because the same action can have different consequences in different states. Searching while high is usually desirable; searching while low carries the risk of battery depletion.

5.2. Video Games

In Atari-style game playing, the state may be represented by recent game frames, the actions are legal joystick/button controls, and the reward is typically the change in game score. A terminal state may occur when the game ends.



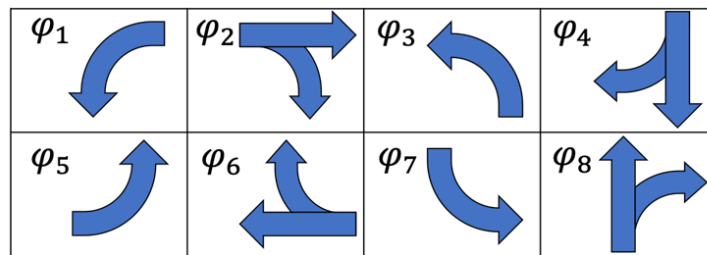
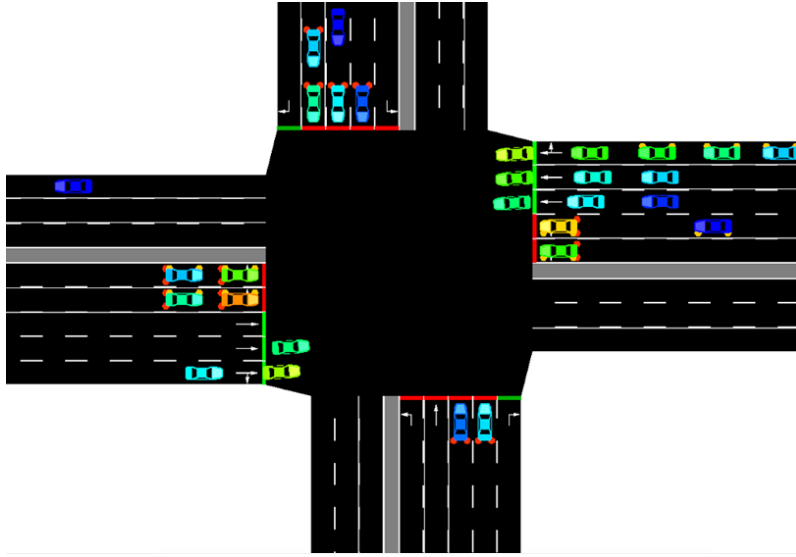
Video game control can be formulated as an MDP with screen-based states, game actions, score rewards, and terminal game states.

MDP element	Atari-style interpretation
State	Recent frames or processed screen representation.
Action	A discrete game control such as left, right, fire, or no-op.
Reward	Score increment or game-specific reward signal.
Transition	The emulator updates the screen and internal game variables after the chosen action.
Policy	A rule mapping screen information to game controls.

This is a finite MDP in principle because screen configurations and action choices are finite, although the state space is extremely large. This motivates function approximation and deep reinforcement learning.

5.3. Traffic Signal Control

A traffic signal controller can also be formulated as an MDP. The state summarizes traffic conditions, the action selects or extends a signal phase, and the reward is designed to encourage efficient traffic movement.



Traffic-signal control: the agent can choose signal phases based on traffic state to reduce delay and queues.

MDP element	Traffic-control interpretation
State	Queue lengths, waiting times, current phase, detector counts, and elapsed green time.
Action	Select the next phase, extend the current phase, or switch to another admissible phase.
Reward	Negative delay, negative queue length, throughput, or a weighted traffic-performance measure.
Transition	Vehicle arrivals, departures, queue changes, and phase changes over the next time interval.
Policy	A signal-control rule choosing actions from observed traffic conditions.

For a finite MDP, continuous measurements may be discretized. The modelling challenge is to choose a state representation rich enough to support good decisions, but compact enough to learn from data.

6. Goals and Rewards

In reinforcement learning, the agent's goal is formalized through the reward signal. The reward is a scalar number supplied by the environment. The agent's objective is to maximize the expected cumulative reward, not to maximize a verbal description of the goal directly.

The **reward hypothesis** says that goals and purposes can be represented as the maximization of expected cumulative reward. Thus, the reward signal should encode what the agent is meant to achieve.

Task	Possible reward	What the reward encourages
Maze navigation	−1 per step until the goal	Reaching the goal quickly.
Recycling robot	Positive reward for cans collected; penalty for battery failure	Search when useful, recharge when necessary.
Video game	Score change	Actions that increase the game score.
Traffic control	Negative total delay or queue length	Reducing congestion and waiting.

Reward design is important. A poorly chosen reward may make the agent optimize the wrong behaviour. In an MDP, however, the reward is still treated as part of the environment and not something the agent is allowed to change arbitrarily.

7. Returns and Episodes

The return is the total future reward from a time step onward. It is the quantity that value functions try to predict.

7.1. Episodic Tasks

In an episodic task, interaction breaks naturally into episodes. Each episode ends at a terminal time T , such as the end of a game or completion of a task. The return from time t can be written as

$$G_t \doteq R_{t+1} + R_{t+2} + \cdots + R_T. \quad (6)$$

For example, in a maze task with reward -1 per step until the exit, a shorter path gives a larger return because fewer negative rewards are accumulated.

7.2. Discounted Return

For continuing tasks, or when future rewards should be weighted less than immediate rewards, the discounted return is

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}. \quad (7)$$

where $0 \leq \gamma \leq 1$. This can also be written recursively as

$$G_t = R_{t+1} + \gamma G_{t+1}. \quad (8)$$

Discount factor	Interpretation
$\gamma = 0$	The agent considers only immediate reward.
$0 < \gamma < 1$	Future rewards matter, but later rewards are discounted.
γ close to 1	Long-term consequences receive strong importance.

Numerical example. Suppose rewards are $R_{t+1} = 2$, $R_{t+2} = 3$, and $R_{t+3} = 5$, with $\gamma = 0.9$. Then

$$G_t = 2 + 0.9(3) + 0.9^2(5) = 2 + 2.7 + 4.05 = 8.75.$$

8. Policies and Value Functions

A policy specifies how the agent behaves. For a stochastic policy, $\pi(a | s)$ is the probability of selecting action a in state s .

The state-value function under policy π is

$$v_\pi(s) \doteq \mathbb{E}_\pi[G_t | S_t = s]. \quad (9)$$

It gives the expected return when the agent starts in state s and thereafter follows policy π .

The action-value function under policy π is

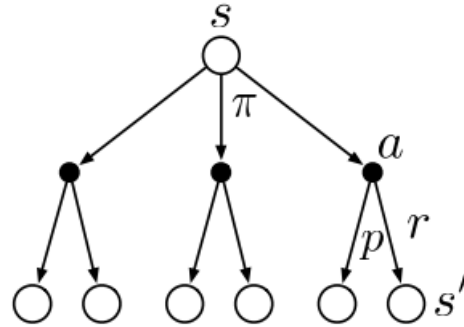
$$q_\pi(s, a) \doteq \mathbb{E}_\pi[G_t | S_t = s, A_t = a]. \quad (10)$$

It gives the expected return after taking action a in state s and then following policy π .

8.1. Bellman Equation for v_π

Using $G_t = R_{t+1} + \gamma G_{t+1}$, the value of a state can be related to the values of its possible successor states:

$$\begin{aligned} v_\pi(s) &= \mathbb{E}_\pi[G_t | S_t = s] \\ &= \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s] \\ &= \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_\pi(s')]. \end{aligned} \quad (11)$$



Backup diagram for v_π

Backup diagram for v_π : the value of s is backed up from possible actions, rewards, and successor states.

This equation says that $v_\pi(s)$ is an average over all actions selected by the policy and all next states and rewards generated by the environment. The backup diagram represents this averaging structure.

8.2. Bellman Equation for q_π

Similarly, the action-value function satisfies

$$q_\pi(s, a) = \sum_{s', r} p(s', r | s, a) \left[r + \gamma \sum_{a'} \pi(a' | s') q_\pi(s', a') \right]. \quad (12)$$

Here, after taking a in s , the environment moves to s' and gives reward r ; then the policy chooses future actions from s' .

9. Gridworld Example

The gridworld example is a compact finite MDP. Each grid cell is a state. The actions are north, south, east, and west. Actions that would move the agent off the grid leave it in the same cell and give reward -1 . Other ordinary moves give reward 0 .



Gridworld example: exceptional transitions from A and B, action directions, and the state-value function under the equiprobable random policy.

The special states have exceptional dynamics. From state A, any action gives reward $+10$ and moves the agent to A'. From state B, any action gives reward $+5$ and moves the agent to B'. Under the equiprobable random policy, each action is chosen with probability 0.25 .

The value table on the right gives $v_\pi(s)$ for $\gamma = 0.9$. The high value of A reflects its immediate reward $+10$, but the value is less than 10 because A sends the agent to A', from which future random movement may cause edge penalties. Values near the lower edge are often negative because random movement has a significant chance of hitting the boundary.

The following two checks use row-column references in the displayed 5×5 grid. Rows are counted from top to bottom and columns from left to right.

Worked case 1: ordinary interior cell at row 3, column 3. The displayed value of this cell is 0.7. Its four neighbouring values are: up $(2, 3) = 2.3$, right $(3, 4) = 0.4$, down $(4, 3) = -0.4$, and left $(3, 2) = 0.7$. For ordinary moves, reward is 0, the policy chooses each action with probability $1/4$, and $\gamma = 0.9$:

$$\begin{aligned}
 v_\pi(3, 3) &= \frac{1}{4} \left[(0 + 0.9 \cdot 2.3) + (0 + 0.9 \cdot 0.4) \right. \\
 &\quad \left. + (0 + 0.9 \cdot (-0.4)) + (0 + 0.9 \cdot 0.7) \right] \\
 &= \frac{1}{4} (2.07 + 0.36 - 0.36 + 0.63) = 0.675 \approx 0.7.
 \end{aligned}$$

Thus the Bellman average is consistent with the value shown for row 3, column 3.

Worked case 2: special state A at row 1, column 2. The displayed value of A is 8.8. From A, every action gives reward +10 and moves to A' at row 5, column 2, whose displayed value is −1.3. Since all four actions have the same reward and same next state, the Bellman relation reduces to

$$\begin{aligned} v_\pi(1, 2) &= \sum_a \frac{1}{4} [10 + 0.9 v_\pi(5, 2)] \\ &= 10 + 0.9(-1.3) = 8.83 \approx 8.8. \end{aligned}$$

This explains the high value of A: the immediate reward +10 dominates the discounted continuation value from A'.

10. Optimal Policies and Optimal Value Functions

A policy π is better than or equal to another policy π' if its expected return is at least as large in every state. An optimal policy is a policy that is not worse than any other policy.

The optimal state-value function is

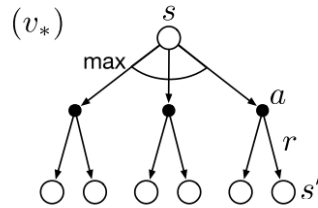
$$v_*(s) \doteq \max_{\pi} v_{\pi}(s), \quad \forall s \in \mathcal{S}. \quad (13)$$

It gives the best expected return achievable from state s over all policies.

10.1. Bellman Optimality Equation for v_*

The optimal value of a state is obtained by choosing the action whose expected one-step reward plus discounted optimal successor value is largest:

$$\begin{aligned} v_*(s) &= \max_a q_*(s, a) \\ &= \max_a \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a] \\ &= \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_*(s')]. \end{aligned} \quad (14)$$



Backup diagram for v_* : the agent chooses the action branch with maximum expected value.

In the Bellman expectation equation, the policy averages over actions. In the Bellman optimality equation for v_* , the policy averaging is replaced by maximization over actions.

The optimal action-value function is

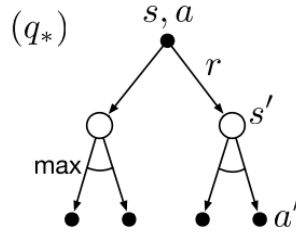
$$q_*(s, a) \doteq \max_{\pi} q_{\pi}(s, a), \quad \forall s \in \mathcal{S}, a \in \mathcal{A}(s). \quad (15)$$

It gives the best expected return after first taking action a in state s and then behaving optimally.

10.2. Bellman Optimality Equation for q_*

For action values, the first action a is fixed. After the next state is reached, the agent chooses the best next action:

$$\begin{aligned} q_*(s, a) &= \mathbb{E} \left[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') \mid S_t = s, A_t = a \right] \\ &= \sum_{s', r} p(s', r \mid s, a) \left[r + \gamma \max_{a'} q_*(s', a') \right]. \end{aligned} \quad (16)$$



Backup diagram for q_* : after the fixed first action, the best next action is selected.

The arc labelled “max” in the backup diagrams represents an optimal choice. These diagrams make clear where averaging is used and where maximization is used.

If v_* is known and the dynamics are known, an optimal action in state s can be selected by one-step lookahead: choose an action maximizing $\sum_{s', r} p(s', r \mid s, a)[r + \gamma v_*(s')]$.

11. Optimality and Approximation

For small finite MDPs, the Bellman optimality equations can in principle be solved exactly. In realistic problems, however, the state space may be extremely large. Video games, traffic networks, robotics, and industrial systems may have so many states that storing exact values for all states is not feasible. In such cases, the goal shifts from exact optimality to useful approximation. The agent may learn approximate value functions, approximate policies, or use sampled experience rather than complete dynamics. This is the bridge from finite tabular methods to approximate methods and deep reinforcement learning.

The central idea remains the same: use rewards, transitions, and value estimates to improve sequential decision making under uncertainty.

12. Review of Key Terms

Term	Short definition
Finite MDP	A Markov decision process with finite state, action, and reward sets.
State, S_t	The information used at time t to make decisions and predict future consequences.
Action, A_t	The decision selected by the agent at time t .
Dynamics, $p(s', r \mid s, a)$	Probability of next state s' and reward r after action a in state s .
Transition probability, $p(s' \mid s, a)$	Probability of next state s' after action a in state s .
Expected reward, $r(s, a)$	Average immediate reward expected from state s after action a .
Policy, $\pi(a \mid s)$	Probability of selecting action a in state s .
Return, G_t	Total future reward from time t onward, possibly discounted.
Discount factor, γ	Parameter controlling the present importance of future rewards.
State-value function, v_π	Expected return from a state under policy π .
Action-value function, q_π	Expected return after taking an action in a state and then following π .
Bellman equation	A self-consistency equation relating a value to expected rewards and successor values.
Optimal value function	The best achievable value function over all policies.
Backup diagram	A graphical representation of how value is backed up from possible future states and actions.

13. Review Questions

1. Define a finite MDP. Explain why the finiteness of $\mathcal{S}$, $\mathcal{A}$, and $\mathcal{R}$ is useful in tabular reinforcement learning.
2. Explain the agent-environment interface using the trajectory $S_0, A_0, R_1, S_1, A_1, R_2, \dots$. Why is the reward following A_t written as R_{t+1} ?
3. State the Markov property in the context of an MDP. What must the state representation contain for this assumption to be reasonable?
4. Given the following dynamics for state s and action a , compute $p(s_1 \mid s, a)$, $p(s_2 \mid s, a)$, and $r(s, a)$.

s'	r	$p(s', r \mid s, a)$
s_1	2	0.30
s_1	0	0.20
s_2	4	0.10
s_2	1	0.40

5. Compute the discounted return for the reward sequence 2, 0, 3, 5 with $\gamma = 0.8$. Then compute it again with $\gamma = 0$ and interpret the difference.

6. A continuing traffic-control task gives reward equal to negative total vehicle delay every minute. Explain why discounting may be useful, and also mention one possible drawback of choosing γ too small.
7. Consider a state s with two actions. Action a_1 gives reward 2 and moves to s_1 with probability 1. Action a_2 gives reward 0 and moves to s_2 with probability 1. If $\gamma = 0.9$, $v(s_1) = 5$, and $v(s_2) = 8$, which action is greedy with respect to these values? Show the one-step lookahead calculation.
8. In the recycling robot example, identify the states, actions, rewards, and one stochastic transition. Why does the transition table define a finite MDP?
9. For the recycling robot, suppose from state *low*, action *search* gives reward $r_{\text{search}} = 4$ and remains in *low* with probability $\beta = 0.6$, but moves to *high* with probability $1 - \beta = 0.4$ with reward -3 . If $v(\text{low}) = 5$, $v(\text{high}) = 8$, and $\gamma = 0.9$, compute the one-step expected return for *search* from *low*.
10. Formulate an Atari game as an MDP. Specify state representation, actions, rewards, terminal condition, and why function approximation becomes necessary.
11. Formulate a traffic signal control problem as a finite MDP after discretization. Clearly specify states, actions, rewards, and transition behaviour.
12. Write the Bellman equation for v_π and explain in words why it averages over both the policy and the environment dynamics.
13. Derive the Bellman optimality equation for v_* from the idea that the optimal value of a state is obtained by the best first action followed by optimal behaviour.
14. In the gridworld example, verify the Bellman relation for special state B using reward $+5$, $\gamma = 0.9$, and the displayed value of B'. Why is the value only approximately equal to the displayed value?
15. Explain the difference between exact optimality and useful approximation. Why is approximation unavoidable in large MDPs such as video games or traffic networks?

14. Points to Remember

- Finite MDPs extend bandits by adding states and state transitions.
- The dynamics $p(s', r \mid s, a)$ fully describe how the finite MDP responds to actions.
- Rewards define the goal; returns accumulate rewards over time.
- Policies specify behaviour; value functions evaluate long-term consequences.
- Bellman equations are self-consistency relationships for value functions.
- Bellman optimality equations replace policy averaging with maximization over actions.
- Exact optimality is possible only in small or tractable settings; large problems require approximation.

15. Required Reading

Sutton, R. S., and Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press. Required reading: Chapter 3, especially Sections 3.1 to 3.7, covering finite Markov decision processes, the agent-environment interface, dynamics, goals and rewards, returns, value functions, Bellman equations, and optimality.

16. References

- Sutton, R. S., and Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). Cambridge, MA: MIT Press.
- Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., and Riedmiller, M. (2013). *Playing Atari with Deep Reinforcement Learning*. arXiv:1312.5602.
- Ault, J., Hanna, J. P., and Sharon, G. (2019). *Learning an Interpretable Traffic Signal Control Policy*.

17. Acknowledgements

These lecture notes are based on and adapted from Chapter 3 of Sutton and Barto's *Reinforcement Learning: An Introduction*, Second Edition.

End of Topic -3